

Dev Log

Keep this updated daily. Short entries are fine — the point is having a real record of decisions and problems for interview answers, not prose.

Day 1

- Provisioned Oracle Cloud compute instance. Had to switch from VM.Standard.A1.Flex (ARM/Ampere) to VM.Standard.E5.Flex (AMD x86) because ARM capacity was exhausted in the Mumbai region — a real, common free-tier constraint. Switching to x86 simplified later steps (Docker/Ollama/k3s all default to x86, no ARM-compatibility flags needed).
- Set up SSH key-based auth, installed Docker (user added to docker group to run without sudo).
- Installed Ollama (systemd service, CPU-only mode — no GPU on this box), pulled llama3.2:1b, confirmed inference works via `ollama run`.
- Built the FastAPI gateway (auth via API key, concurrency-limited request queueing via `asyncio.Semaphore`, Prometheus metrics, `/health` endpoint) and ran it directly with `uvicorn` against local Ollama — confirmed a full end-to-end response via `curl`.

Day 2

- Load-tested the concurrency limiter with Apache Bench (`ab`): 20 requests, 5 concurrent, against a limiter capped at 2 — zero real connection failures (`ab`'s "Failed requests" count was a false positive caused by variable-length LLM responses, not real errors).
- Added contract tests (health status contract, validation rejection, metrics counter actually incrementing) beyond basic smoke tests.
- Found via a failing test that FastAPI's `APIKeyHeader` returns 403 for a missing auth header vs 401 for a present-but-wrong key — fixed the test to match real behavior instead of an incorrect assumption.
- Built and ran the gateway as a Docker container for the first time (`--network host` for local testing), confirmed it still worked identically containerized.

Day 3

- Installed k3s (lightweight Kubernetes) as a single-node cluster on the Oracle box.
- Fixed a real permissions issue with `kubect1` config access (`KUBECONFIG` env var, persisted to `~/.bashrc`).
- Built the gateway's Docker image locally and imported it directly into k3s's container storage (`docker save | k3s ctr images import -`) — a temporary

shortcut for local testing; Day 4's CI pipeline replaces this with a proper registry (GHCR).

- Created a Kubernetes Secret to hold the API key (plaintext/base64 at this point — Day 6 fixes this properly with Sealed Secrets).
- Deployed both Ollama and the gateway as Kubernetes Deployments, each with their own Service.
- Watched the gateway self-heal automatically (one restart) because its health check correctly detected Ollama wasn't ready yet — a live demo of Kubernetes' self-healing behavior.
- Discovered the new Ollama pod needed its model pulled fresh, since container storage doesn't carry over from the old systemd-based setup — reinforced that containers are ephemeral unless given persistent storage.
- Confirmed full request flow end-to-end using Kubernetes-internal networking (<http://ollama:11434>), not `localhost` shortcuts.

Day 4

Built the GitHub Actions CI pipeline (lint → test → build → Trivy scan → push to GHCR → bump k8s manifest). Real issues hit and fixed:

1. Trivy found 3 real HIGH-severity CVEs in `starlette` (transitive dependency via `fastapi`) — fixed by loosening the `fastapi` version pin so `pip` resolves a patched version automatically.
2. A pinned Trivy action version (`@0.24.0`) had been deleted/renamed upstream — fixed by switching to `@master`.
3. GHCR requires lowercase image names — GitHub repo names can have capitals, so added an explicit lowercase-conversion step and passed it between jobs via GitHub Actions' job outputs (env vars don't carry across separate jobs).
4. A flake8 style rule caught a missing trailing newline.
5. A FastAPI version upgrade changed which exact HTTP status code (401 vs 403) fires for a missing auth header — fixed the test to assert the actual security property (rejection) rather than the exact status code.

Day 5

- Installed Argo CD (all 7 components healthy), connected it to the GitHub repo watching `main`.
- Real bug: Argo CD's Application manifest specified a different namespace (`undercroft`) than where Day 3's manual `kubectl apply` had put things (`default`) — ended up with two duplicate copies of the whole app running side by side. Diagnosed via `kubectl get pods --all-namespaces`, fixed by recreating the `gateway-secrets` Secret in the new namespace and deleting the old orphaned `default`-namespace resources.

- Proved the full GitOps loop live end-to-end: code push → CI builds/tests/scans/pushes image → Argo CD auto-detects the change → auto-deploys → confirmed with a real curl response. Captured this as a screen recording.

Day 6

- Installed the Sealed Secrets controller and the `kubeseal` CLI.
- Re-created the API key as an encrypted SealedSecret (encrypted with the cluster's public key via kubeseal, only decryptable by this specific cluster's private key), committed it to git, deleted the old plaintext Secret.
- Confirmed the full request chain still works with the new encrypted secret.
- Real claim: the only version of the API key that exists in git history is encrypted — never plaintext, never committed unencrypted.
- Known trade-off: SealedSecrets are cluster-specific — rebuilding the cluster from scratch would require re-sealing, since a fresh install generates a new private key.

Day 7

- Installed Prometheus + Grafana via the kube-prometheus-stack Helm chart into the undercroft namespace.
- Created a ServiceMonitor (gateway-metrics) to tell Prometheus to scrape the gateway's /metrics endpoint every 15s.
- Real bug: Prometheus showed "0/0 up" / "No targets" for gateway-metrics even though the metric existed (confirmed via direct curl to /metrics) and the Service had real Endpoints. Root cause: a ServiceMonitor's `selector.matchLabels` matches a Service's own `metadata.labels` — not its `spec.selector`, which only controls pod routing. The gateway Service had a selector but no labels of its own, so Prometheus couldn't discover it via service discovery even though it worked fine for normal traffic.
- Diagnosed by: checking raw metrics at the source -> inspecting Prometheus's live scrape config via /config -> checking the Service's Endpoints (ruled out networking) -> isolating the label mismatch.
- Fix: added `metadata.labels: app: gateway` to the gateway Service (separate from spec.selector, which was already correct) — one line, auto-deployed via Argo CD once pushed.
- Confirmed fix via Prometheus's own Targets UI: gateway-metrics -> 1/1 up.
- Built a Grafana dashboard ("Undercroft Gateway") with a live "Request Rate" panel (`rate(gateway_requests_total[5m])`) and a Queue Depth panel, showing real traffic from test curls.

- Local dev access pattern: `kubect` port-forward for Prometheus (9090), Grafana (3000), and the gateway (8080), each tunneled out to Windows via SSH -L flags — up to 4 persistent terminals during active dev/testing.

Day 8

- Real bug found: CI's `update-manifest` job used a `sed` pattern that only matched lines starting with `image: ghcr.io/` — but `deployment.yaml` still had `image: gateway:local` (the Day 3 local test image) with `imagePullPolicy: Never`. Every CI run since Day 4 had been silently doing nothing to that line — reported success (green checkmarks) while never actually bumping the deployed image. A good example of "the pipeline passing" not being the same as "the pipeline doing what you intended."
- Fixed by manually pointing `deployment.yaml` at the real GHCR image and removing `imagePullPolicy: Never`. This also fixed the `sed` pattern match going forward, so future CI runs now correctly bump the tag on every push.
- Confirmed a real zero-downtime rollout: new gateway pod pulled the GHCR image, went Running/Ready in ~16s, and only then was the old pod terminated — `maxUnavailable: 0 / maxSurge: 1` working as designed.
- Verified PVC persistence for Ollama: deliberately deleted the Ollama pod, confirmed the replacement pod came up and served a real inference request immediately with no re-pull needed — model data genuinely persisted on the `PersistentVolumeClaim`. (Earlier re-pulls on Day 3/5 were explained: those were brand-new pods in different namespaces during migration, not a persistence bug.)
- Load tested with real numbers: 50 requests, 5 concurrent, limiter capped at 2 — median latency ~14s, max ~15-16s, zero real dropped connections. Ran a controlled comparison (full monitoring stack vs. monitoring scaled to zero) to test whether Prometheus/Grafana overhead was contributing to latency — result: no meaningful difference (14.05s vs 13.8s median). Conclusion: the bottleneck is CPU-bound LLM inference itself on a CPU-only box under concurrency limiting, not observability overhead. This is a stronger, more honest answer than assuming monitoring was the cause — it came from an actual controlled test.

Day 9